Programming Projects

Include both a report and the source code of the project. A github repo is usually easiest for me (make sure it's public).

Please clearly label your report – include the text of the question in your file. Unlike the homework, project reports should be written in grammatical sentences and contain an introduction, results, and conclusion/discussion. Make sure to include enough results to justify your conclusions, but do not include figures that you don't discuss. The discussion should include a general description of the salient points of the result, as well as answering any specific questions. Just code is not sufficient.

You may use any language: python, Perl, Matlab, R, C, C++, Java, etc. Make sure to include sufficient internal or external documentation for me to understand how the program works. Command line, interactive, or GUI input are acceptable. I have Unix and Windows platforms available. My preference would be for command line input and Unix OS.

Your solution should include

- Explanation of your approach, implementation, and results in well organized and grammatical text.
- source code
- instructions to install and run the code. Be sure to include necessary dependencies and packages.
- test data and expected results
- Answers to any specific questions

You may use existing packages/libraries to read and write sequences, but, unless otherwise instructed, generate your own code. Do not use solutions produced by LLM or that you look up on the internet.

PP1: Alignments DNA+Protein alignment

40 pts-55 pts

Align a DNA sequence and a protein sequence based on the translated sequences of the DNA sequences. Only the forward reading frames need be considered. If desired, you may use existing packages for reading/writing sequences, scoring tables, and formatting the results. Display the results in conventional pairs of lines format, i.e., in blocks with the aligned sequences above each other, showing both the DNA and translated protein sequences.

- Use a local alignment algorithm
    - Bonus: +10 pts. Implement both global and local alignments
- Assume the sequences are in fasta format
- Use a scoring table such as Blosum or PAM read in from a file. A good source of scoring matrix files is ftp://ftp.ncbi.nlm.nih.gov/blast/matrices/
- Allow affine gaps within reading frames
- Allow length independent gaps between reading frames.
    - Bonus + 5 pts. : allow affine gaps between reading frames

PP2: Suboptimal alignment

30 pts

For DNA or protein sequences, report *n* non-intersecting suboptimal alignments. As discussed in class, sub-optimal alignments use no pair of letters used in earlier alignments. This can be achieved by recalculating the score matrix, setting the score for all letter-pairs in previous alignments to be zero.

- allow selection of an appropriate scoring table for DNA or protein read from a file. A good source of scoring matrix files is ftp://ftp.ncbi.nlm.nih.gov/blast/matrices/
- use a local algorithm with affine gap penalties
- display aligned sequences in conventional pairs of lines format, , i.e. in blocks with the aligned sequences above each other

PP3: Sequence randomization

20 - 40 pts

- . Randomize a sequence preserving its k-mer word content (i.e., 1 letter, 2-letter, 3-letter, …). This may be solved simply using sampling with or without replacement, but the k-mer content of the randomized sequence will not exactly match the original sequence. A more complicated approach is to provide an exact solution using Euler's method.
- input and output in FastA format. Your program should work for any alphanumeric letter sequence, but especially DNA and protein alphabets.
- k-mer should be selectable in range 1 to 6 (or more)
- Additional output should show original and randomized k-mer word content
- Bonus: 20 pts. Implement both a sampling solution and an exact solution (all k-mer counts exactly the same as in the original sequence) using Euler's algorithm.

PP4: Sequence evolution

40 pts each for DNA and protein models

- A zero order Markov model (no dependency between positions or i.i.d. model) can be generated from the frequencies of the letters in DNA or protein sequences. It is widely believed that a simple i.i.d. model is adequate for proteins, but DNA requires a much higher order model. Such higher order models give the probability of each letter given the previous N positions, where N is the order.
- Using real sequences, calculate order 0 -3 models for proteins or 0-4 for DNA. Make sure that the sequences you use to tabulate frequencies are not highly similar (<50% for DNA, < 30% for protein).
- Use these models to generate sequences according to the Markov models.
- Good models should generate kmer word frequencies that are similar to real sequence, i.e. the models are capable of generating protein-like or DNA-like sequences. This can be tested using a chi^2 test (see for instance, Chen, W., Kenney, T., Bielawski, J. et al. Testing adequacy for DNA substitution models. BMC Bioinformatics 20, 349 (2019). https://doi.org/10.1186/s12859-019-2905-3)
- If the chi^2 does not allow us to reject the hypothesis that the observed and generated come from the same distribution, we will say that the model is sufficient to generate protein-like or DNA-like sequences.
- Scoring:

- 10 pts. selection of training sequences: document the sequences you choose and the degree of similarity. Justify the number of sequences you select, and their independence.
- 15 pts. Generation of markov models of order 0-3 or 0-4 for protein and DNA, respectively.
- 15 pts. Statistical tests comparing real and generated data. Consider how many replicates you need.

PP5: Evolution/Trees (40 – 55 pts)

- Generate a series of sequences related by a simulated evolutionary process. Starting with a protein sequence (at least 100 residues), apply a random mutation process based on the PAM-1 model. At random times, duplicate the sequence creating speciation events. At random times, remove some species (extinctions). This is your known tree, you will know the exact relationship (true tree) relating your sequences, and the branch lengths (number of mutations between sequences).
- Input parameters should include total evolution time (% true mutations), and number of leaf sequences to be generated.
- speciation and extinction events can follow simple a simple probability, i.e., probability of speciation or extinction at each step is constant.
  - bonus: 15 pts. implement a more sophisticated model where these events follow a distribution such as normal or gamma
- report the final set of sequences as a multiple alignment. No alignment algorithm is necessary, since the sequences are generated by random mutation, they are already aligned.
- report the tree in Newick format.
- report the true number of mutations between each pair of sequences (from the simulation history) and the observed number of differences (n x n table, where n is number of leaf nodes)

PP6 Parsimony/UPGMA Trees (40 - 85 pts)

6.1 Parsimony (40 pts)

This project requires solving Evolution/Trees, PP5, above.

- Implement the parsimony algorithm as described in class.
- for 50 random trees with 20 leaf nodes, calculate the tree lengths and report the true tree lengths (known from the simulation), estimated tree length (from parsimony). Calculate the mean and standard deviation of the difference between the known and estimated tree lengths.
- Repeat for evolutionary distances of 50%, 100%, 200%. Are there significant differences between the known and estimated trees?

6.2 UPGMA (30-45 pts)

requires solving Evolution PP5, above

- Do the same as above for UPGMA trees. - requires solving Evolution #1, above.
  - use 50 random trees with 20 leaf nodes
  - Implement UPGMA
    - Bonus: 15 pts. Use the Fitch-Margoliash method to estimate branch lengths

- Use the observed mutational distances to construct trees, and compare the estimated distances from the tree to the known true distances from the simulation.
- Calculate the mean and standard deviation of the RMS difference between the known and estimated numbers of mutations.
- Repeat for evolutionary distances of 50%, 100%, 200%. Are there significant differences between the known and estimated trees?

P7. Tree Arrangement – 50 - 75 pts

- Given a tree, such as one from P5, or P6 write a program rearrange the tree by rotating around internal nodes so that the distance between adjacent terminal nodes is minimized.
- Use at least 50 taxa
- Use a Blosum62 scoring table to calculate distances
- Report trees before and after rearrangement (draw trees), and sum of the pairwise distances between leaf nodes. Do enough replicates, probably at least 20, to compare the mean and standard deviation between the original and rearranged and show that the distance is significantly decreased.
- You could use a genetic algorithm (GA) for this optimization, but you can use any approach you want.
- +25 pts, compare a greedy approach to a GA approach

P8. De Bruijn Graph – 100-130 points

50 points - Provide a program that

- Generate a random DNA text (parameters: letter frequencies, length). Bonus: 15 points, simulate addition of repetitive sequences(parameters, length, number of repeats)
- Simulate a sequencing project that generates reads of a fixed length (parameters: read_length, coverage, error rate)
- From the reads, generate kmers (parameters: kmer length)
- from the kmers, construct the De Bruijn graph and trace the resulting contigs, breaking the contigs at ambiguous connections. Provide a function to report the sequence fragments and connections between them

  bonus: 15 pts plot a graph of the sequences and their connections. More points for easier to read/understand graphics

  50 points – testing
  - Generate and assemble test sequences of varying lengths and assemble. Do sufficient repetitions to allow the calculation of robust average values. Compare and discuss the number of contigs to the number of contigs predicted by the Lander-Waterman equation.
  - Does error rate affect the accuracy of the Lander-Waterman prediction?

P9. Regex sequence motifs – 100 points

Given a multiple sequence alignment, and a user defined sets of equivalent letters determine a regular expression that, in comparison to a randomized set of sequences, will identify all of the sequences in the original alignment. Deliverables

Code 70 pts:

- generate test and training sequences from the original alignment. The training data should be a subset of the input multiple alignment (parameter: fold, e.g., 0.8), the testing data is the remainder. For the testing data, generate an equal number of randomized sequences with the same distribution of composition (i.e., scramble each individual sequence, not sample with replacement from the letter frequency distribution).
- Identify minimum length regular expressions that correctly classify the training and test data
    - Character classes must correspond to the user defined equivalence classes
    - use cross validation to avoid overtraining. Report the cross validated precision, recall, and F1
    - use any method you choose for optimization: EM, Gibbs sampler, and genetic. algorithms come to mind, but you are not limited.

  Testing 30 pts : Test with sufficient replicates to generate robust average values
- What is the effect of the number/fraction of sequences in the training set used on the precision, recall, and length of the regular expression?
- Is the result using equivalence classes:[ILMV], [DENQ], [HKR], [ST}, [AG], [FWY], C, P better in terms of precision, recall, minimum length, and size of training set required?

P10. Generate potential transcripts – 75 points

Given a DNA sequence identify ORFs on the forward strand. Assuming that introns can be removed at sequences beginning AG and ending GT (inclusive). Identify the spliced sequence with the longest open reading frame. Note that the splicing may change the reading frame when you cross the splice junction – make sure the ORF contains no stop codons. Generate the final amino acid sequence of the longest ORF.

Testing. Choose real DNA sequences containing introns and generate the longest ORFS. Match the ORF sequence to the gene sequence (Blast or MSA). What fraction of predicted ORF is correct? What are the number and length of extra regions at the end (UTRs) and internally (retained introns). Test on at least 10 real genes.